Lecture 5 | MSc Data Handling and Infrastructure for AI

Exploratory Data Analysis with pandas, NumPy and scikit-learn

From a raw table to justified pipeline decisions

Dr Abdul Wahid | SETU | 2026

891
passengers

12
variables

1
leakage-safe pipeline

Today's roadmap: from raw table to pipeline rules



1

Context and structure

What does a row mean? What is in the file?



2

Audit data quality

Missingness, duplicates, validity, outliers



3

Describe and compare

Univariate, bivariate, multivariate, Simpson



4

Split, leak, preprocess

Leakage, NumPy, ColumnTransformer, Pipeline



5

Report and lab

Reproducible EDA report, Lab 5

Running example: Titanic, 891 passengers and 12 columns, so we can focus on reasoning

Learning outcomes

Frame

Design an EDA around a task, population and unit of analysis.

Audit

Detect schema, missingness, duplicate, invalid and unusual values.

Choose

Match visual and numerical methods to variable types and questions.

Challenge

Identify leakage, dependence, sampling bias and distribution shift.

Implement

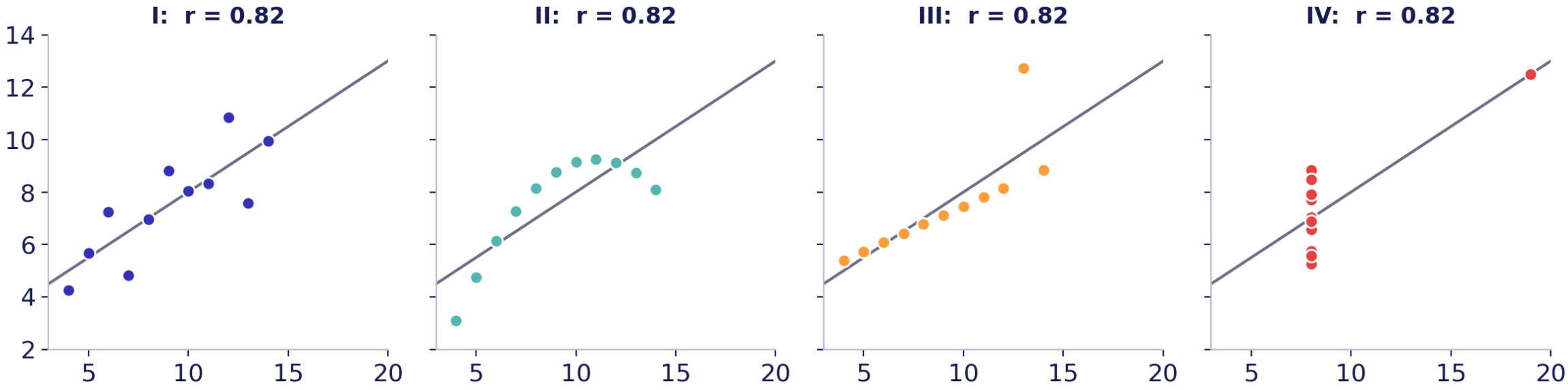
Use pandas and NumPy for analysis and scikit-learn for leakage-safe preprocessing.

Document

Turn findings into explicit pipeline rules and a reproducible report.

The final output is a set of justified decisions, not a gallery of charts

Why EDA? Four datasets, identical statistics



9.0
mean of x

7.50
mean of y

0.82
correlation r

y = 3.0 + 0.5x
fitted line

Same for all four panels (Anscombe, 1973). Only a plot reveals curvature, an outlier, and a single leverage point.

Summary statistics compress data; they cannot tell you what was lost

From statistical foundations to EDA questions

Week 4 concept	EDA question it becomes	Titanic example
Estimand and estimator	What quantity does this summary represent, and for whom?	Mean age 29.7 is the mean of 714 observed ages, not of 891 passengers
Centre, spread, uncertainty	Is the distribution well described by these numbers?	Fare: mean 32.20 but median 14.45 (skewed)
Effect size, conditioning	How large is the difference within groups?	Survival 63% in 1st class vs 24% in 3rd
Missingness, i.i.d.	Who is missing, and are rows independent?	Age missing for 28% in 3rd vs 14% in 1st; 344 passengers share a ticket

EDA operationalises statistical reasoning inside the AI data lifecycle

What EDA is: five verbs, one cycle

Describe
What is present?

›

Diagnose
What is wrong?

›

Compare
What varies by group?

›

Challenge
What could mislead us?

›

Decide
What must the pipeline do?

Exploratory

- Generates questions and hypotheses
- Flexible, many looks at the data
- Findings are provisional

Confirmatory

- Tests pre-specified hypotheses
- Fixed analysis plan, controlled error rate
- Needs data not used to form the hypothesis

EDA produces evidence and hypotheses; it does not prove causal explanations

An iterative EDA workflow

1

Context

Task, population, unit of analysis

What does one row mean?

2

Structure

Schema, types, keys, dimensions

Does the file match the contract?

3

Quality

Missing, duplicate, invalid, extreme

How much, and who is affected?

4

Patterns

Distributions, groups, interactions

What varies, and how certain are we?

5

Actions

Pipeline rules, risks, open questions

What changes, and why?

Iterate: a pattern often raises a new quality or context question

Measure before you clean; record what changed and why

Pause and think

Scenario

A hospital sends you visits.csv with 40,000 rows and asks for a model that predicts 30-day readmission.

Write down five questions you must answer before you plot anything.

Think on your own. If you are watching the replay, pause the video now. Then compare your list with mine.

Keep your list: we will compare it with the workflow in a few minutes

1 Context and structure

Define what one row means, then check what the file actually contains.

Start with the analytical context

Define before coding		Titanic example
Task What decision or prediction is needed?	›	Predict recorded survival (binary)
Population Where should conclusions apply?	›	Passengers in this file, not all voyages
Unit of analysis What does one row represent?	›	One passenger record
Availability When is each feature known?	›	Features must precede the outcome

A vague question produces an unfocused EDA and ambiguous conclusions

Build a data dictionary: a dtype is not a role

Field	Stored as	Analytical role	Trap to record
PassengerId	int64	Identifier (row key)	Never average it; check uniqueness
Survived	int64	Target, binary	Outcome: known only after the event
Pclass	int64	Ordered category	Gaps between 1, 2, 3 need not be equal
Age	float64	Numerical, 19.9% missing	Missing is not random (see section 2)
Ticket	object	Group identifier	344 passengers share a ticket: dependence
Fare	float64	Numerical, right-skewed	May be a ticket total, not a personal price
Cabin	object	Sparse categorical	77% missing, and missingness tracks class

df.dtypes reports storage; a data dictionary records meaning, role and availability

Schema inspection before plotting

```python
df = pd.read_csv('titanic.csv')

print(df.shape)
display(df.head(3))
display(df.dtypes)

# executable data contract
assert df['PassengerId'].is_unique
assert set(df['Survived']) <= {0, 1}
assert set(df['Pclass']) <= {1, 2, 3}
```

Output

```
(891, 12)

   Survived  Pclass     Sex   Age   Fare
0         0       3    male  22.0   7.25
1         1       1  female  38.0  71.28
2         1       3  female  26.0   7.92

PassengerId      int64
Survived         int64
Pclass           int64
Age            float64
Fare           float64

Assertions: passed
```

Assertions prove only what was tested: derive rules from the dictionary, not from the data

Data quality has five dimensions

Completeness

Are required values present?

Age 19.9% and Cabin 77.1% missing

Validity

Do values satisfy domain rules?

15 fares are exactly 0: valid by rule, suspicious by meaning

Uniqueness

Are keys or entities repeated?

0 duplicate rows, yet 344 passengers share a ticket

Consistency

Do fields agree?

Title 'Master' should imply a boy: check Name against Age and Sex

Timeliness

Is it current for deployment?

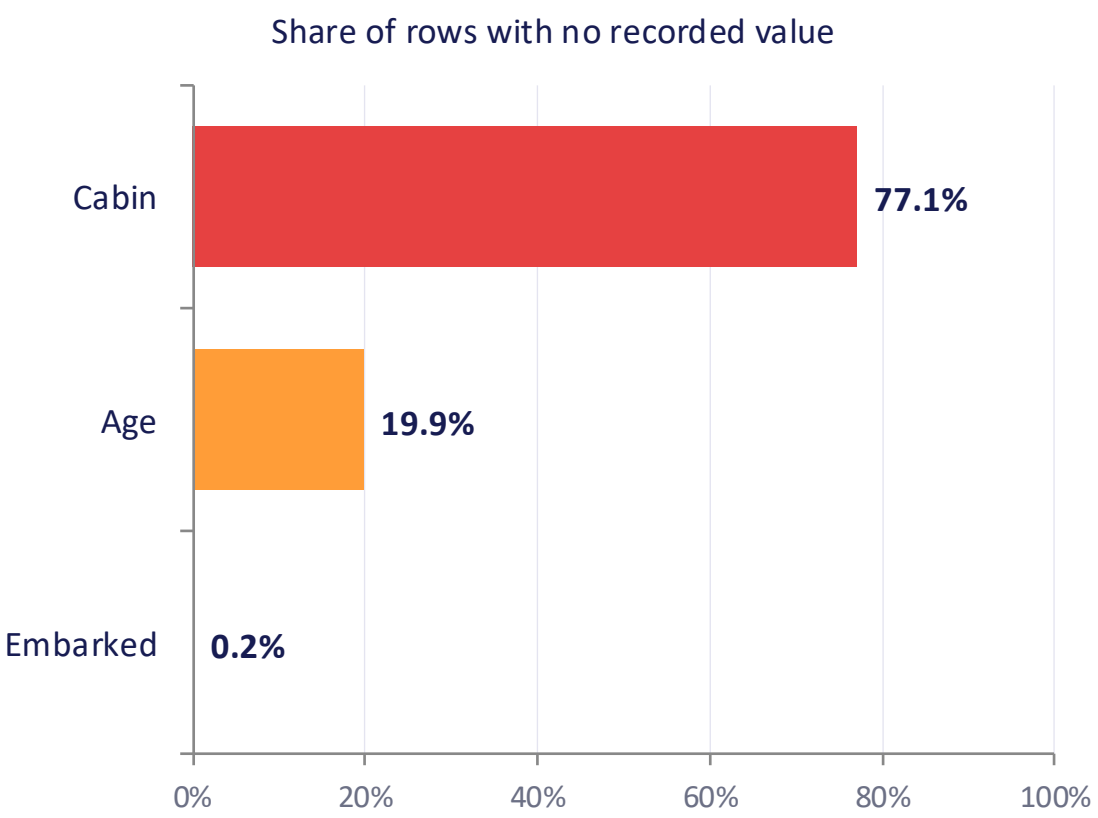
Collected in 1912: fine for teaching, not for a modern model

Cleaning without a quality definition silently changes the population

2 Auditing data quality

Measure before you clean: missingness, duplicates, validity and extreme values.

How much is missing, and where?





Percentages say how much; the matrix shows that missingness has structure

Why is it missing? MCAR, MAR and MNAR

MCAR — **Missing completely at random**

Meaning

Missingness is independent of observed and unobserved values.

Example

A register page is lost by chance.

EDA signal

Missing rate flat across groups

MAR — **Missing at random**

Meaning

Missingness depends only on observed variables.

Example

Age is recorded less often in 3rd class (28% vs 14%).

EDA signal

Rate varies by an observed group

MNAR — **Missing not at random**

Meaning

Missingness depends on the missing value itself.

Example

Older passengers are less willing to state their age.

EDA signal

Not testable from the observed table

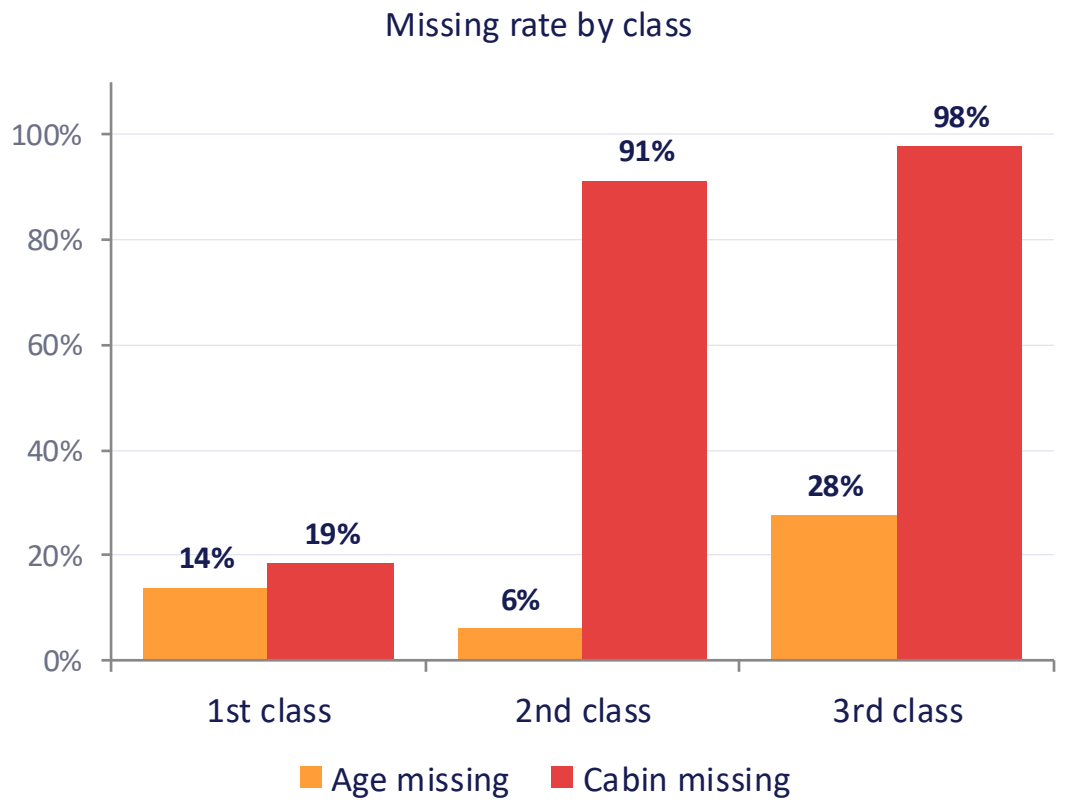
EDA can make MCAR implausible; it can never prove MAR, because data cannot reveal what it lacks

Missingness structure by passenger class

```
miss = (df.assign(age_na=df.Age.isna(),
                  cabin_na=df.Cabin.isna())
          .groupby('Pclass')[['age_na', 'cabin_na']]
          .mean().round(3))
display(miss)
```

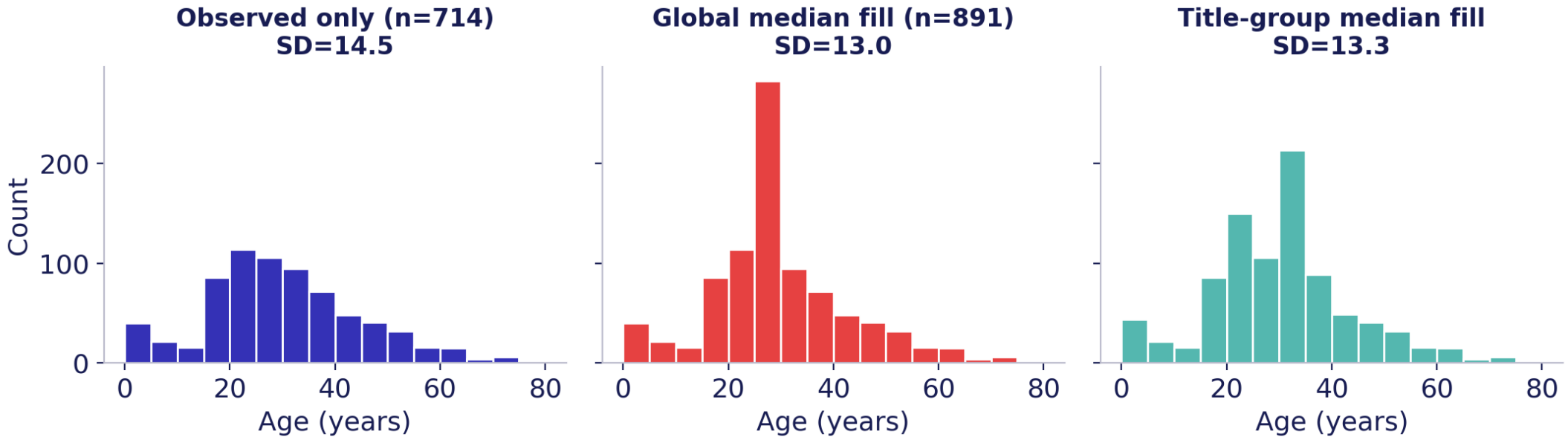
Output

Pclass	age_na	cabin_na
1	0.139	0.185
2	0.060	0.913
3	0.277	0.976



Rates differ by class, so MCAR is implausible, but this alone does not prove MAR

Imputation reshapes the distribution



Global median fill

177 values stack at 28. The SD shrinks from 14.5 to 13.0 and correlations weaken.

Group-aware fill

Title medians differ: Master 3.5, Miss 21, Mrs 35, Mr 30. Better structure, but still a model.

Good practice

Fit the imputer on training rows only. Consider a missing-indicator when missingness is informative.

Imputation is a modelling decision: it changes variance and relationships, not just row counts

Duplicates depend on the unit of analysis

891

passenger rows

681

distinct ticket numbers

344

passengers (39%) share a ticket

Exact duplicate rows

Every field repeats.

Usual cause: ingestion, concatenation, many-to-many join.

Detect: df.duplicated().sum() returns 0 here.

Action: drop, after checking the key.

Entity or event repetition

The same entity legitimately appears more than once.

Examples: a family on one ticket; one patient with many visits.

Action: define the stable key and the grain, then split by group.

Define the entity and stable key before deleting repeated records

Validate with explicit, logged rules

Rule	Rationale	Violations	Response
PassengerId is unique	Row key	0	None needed
Age between 0 and 100	Plausible human range	0	Keep
Fare is at least 0	Prices cannot be negative	0	Keep
Pclass in {1, 2, 3}	Data contract	0	Keep
Fare equals 0 (plausibility)	Free passage? Crew? Unknown	15	**Flag and investigate the source**

6 / 5 / 4

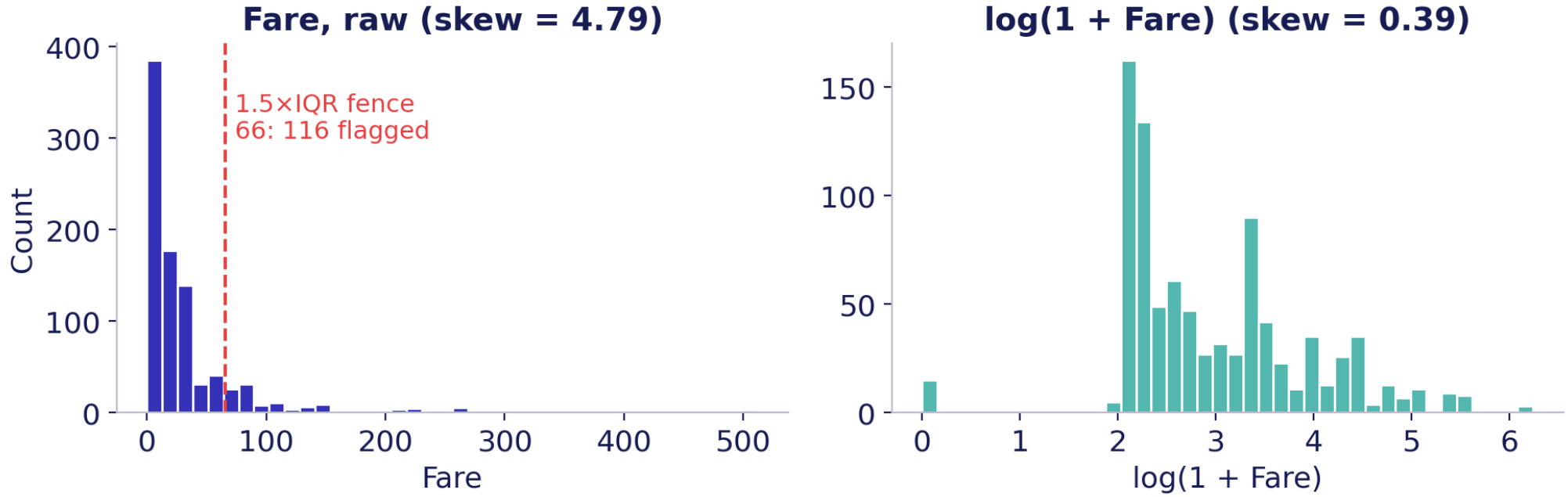
zero fares in 2nd / 1st / 3rd class

1 of 15

zero-fare passengers survived (6.7%)

Record the rule, its rationale, the number of violations and the chosen response

Extreme values: investigate before you delete



32.20
mean fare

14.45
median fare

512.33
maximum fare

116
flagged by the 1.5 IQR rule (13%)

A rarity rule flags candidates; domain knowledge decides what they are

Five kinds of unusual value, five responses

Entry error	Impossible or inconsistent value	**Correct from source or reject with evidence**
Valid extreme	Rare but plausible observation	**Keep; use medians, quantiles or a log scale**
Different population	Generated by another process	**Stratify or redefine the analytical scope**
Influential case	Strong effect on a fitted model	**Refit with and without; check leverage**
Deployment shift	Valid but absent from training	**Monitor ranges and category domains**

Outlier detection is not influence: an extreme value can be harmless, a moderate one decisive

Discussion: is HasCabin a legitimate feature?

67%

survived when a cabin was recorded (n = 204)

30%

survived when cabin was missing (n = 687)

Hint: ask who could report a cabin number after the sinking. Cabin was missing at very different rates by class:

18.5%

Cabin missing, 1st class

91.3%

Cabin missing, 2nd class

97.6%

Cabin missing, 3rd class

Vote in your head

A Drop Cabin: 77% is too much

B Impute the most common deck

C Keep a HasCabin indicator

D Ask when Cabin was recorded

Informative missingness can be a proxy for class, collection process, or the outcome itself

3 Describe and compare

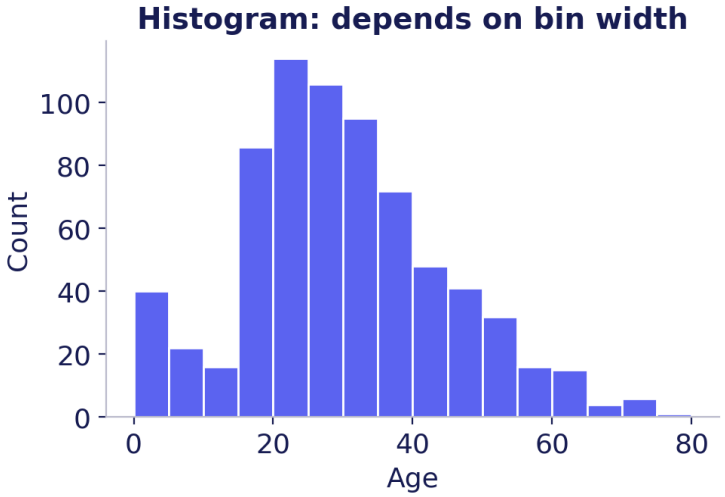
Choose the method from the variable type and the question, then report counts and uncertainty.

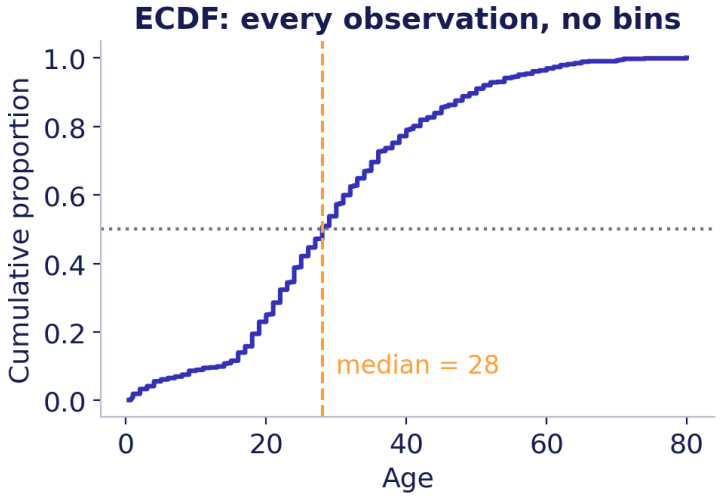
Univariate analysis follows the variable type

	Type	Summarise	Display	Titanic example
	**Numerical**	Centre, spread, quantiles, shape	Histogram, ECDF, box or violin	Age, Fare
	**Categorical**	Counts, proportions, rare levels	Ordered bar chart	Embarked, Pclass
	**Text**	Length, patterns, extracted tokens	Token or pattern counts	Name to title (Mr, Mrs, Master)
	**Time**	Trend, seasonality, gaps, changes	Ordered line plot	Not in Titanic; typical in logs
	**Identifier**	Uniqueness and repetition	Diagnostic table, never a mean	PassengerId, Ticket

The display must respect the variable's meaning and measurement scale

The ECDF: a bin-free view of a distribution






```
age = np.sort(df['Age'].dropna().to_numpy())
ecdf = np.arange(1, len(age) + 1) / len(age)
plt.step(age, ecdf, where='post')
```

Definition

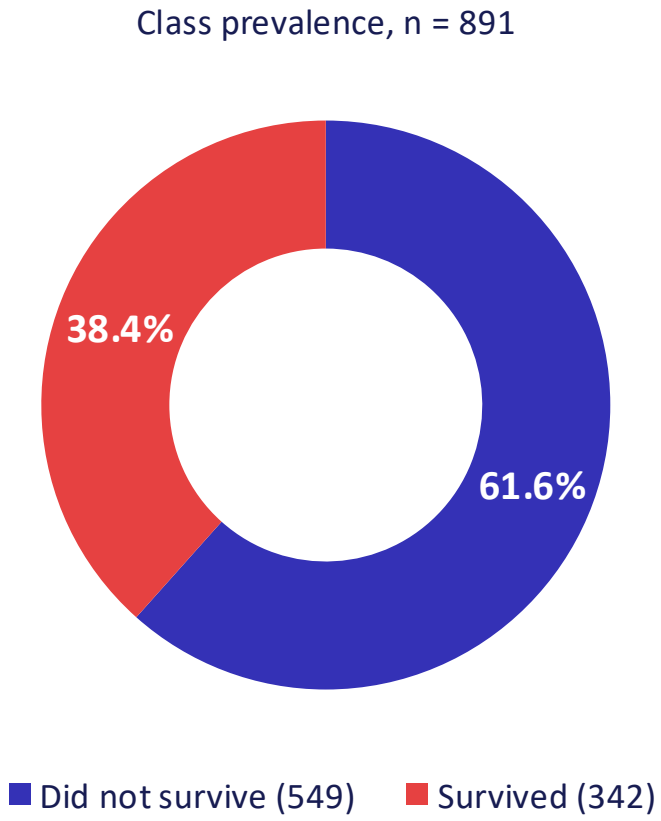
$$F_n(x) = (1/n) \Sigma I(X_i \leq x)$$

Reading it

- F(28) is about 0.50: half the observed ages are 28 or younger.
- Steep: many observations. Flat: a gap.
- n = 714: the 177 missing ages are not in the denominator.

An ECDF keeps every observation and removes the bin-width choice

The target distribution defines the baseline



61.6%

accuracy of a model that always predicts 'did not survive'

- Report prevalence with counts and the denominator.
- Accuracy hides failure on the minority class: also report precision, recall, F1, ROC-AUC and calibration.
- Prevalence can shift at deployment, moving thresholds and predictive values.

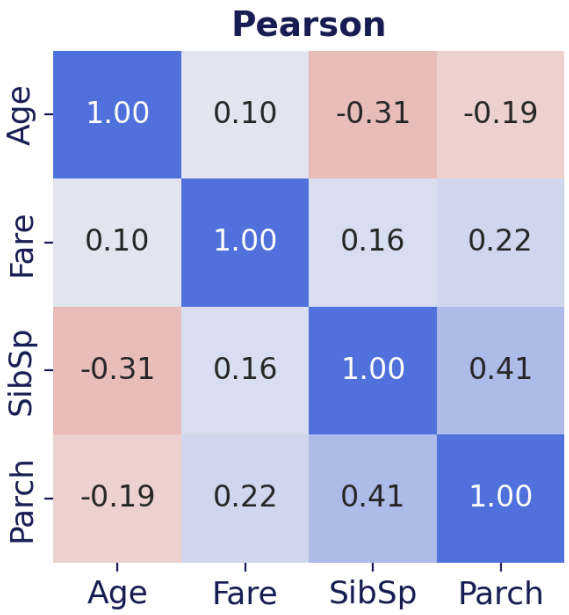
Any model must beat the majority baseline before accuracy means anything

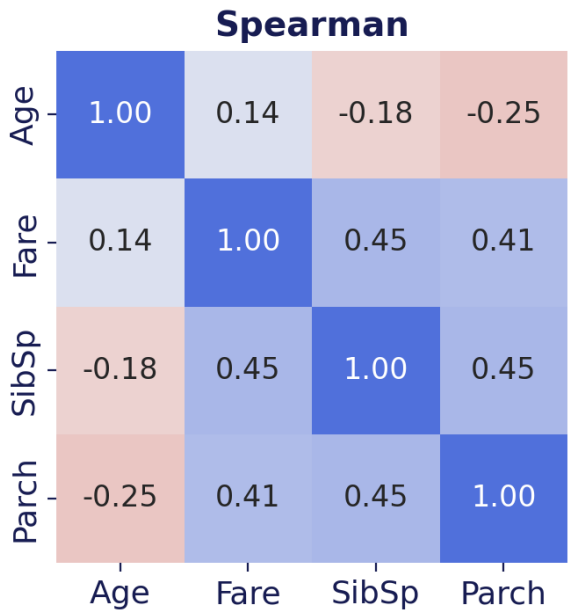
Choose bivariate methods by variable types

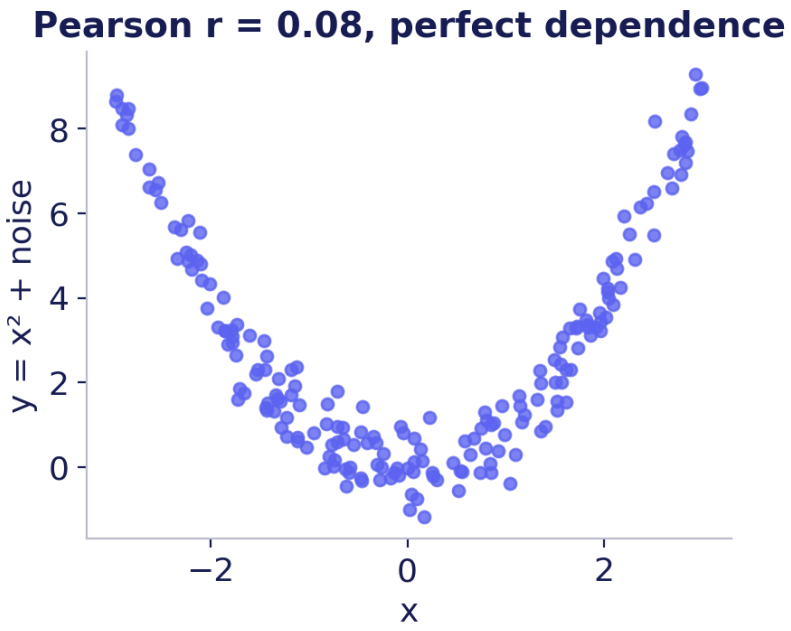
Pair	Question	Methods	pandas / SciPy
Numerical + numerical	Do values move together?	Scatter, Pearson, Spearman	df.corr(method=...)
Categorical + numerical	Do distributions differ by group?	Grouped quantiles, violin, effect size	groupby().agg()
Categorical + categorical	Are proportions associated?	Cross-tab, chi-square, Cramer's V	pd.crosstab, chi2_contingency
Feature + target	Could it support prediction?	Target rate by level, leakage check	groupby(f)[y].mean()
Time + variable	Does the process change?	Ordered trend, rolling window	rolling(), resample()

The question and the measurement scale select the method, not habit

Correlation: inspect before you summarise







Pearson

Linear association; sensitive to outliers and skew.

Spearman

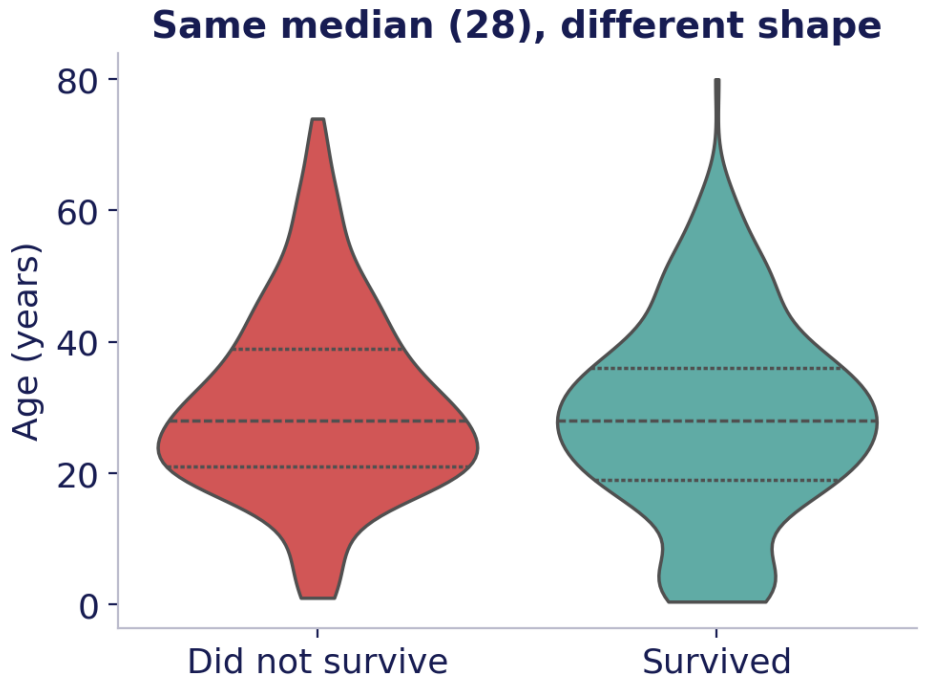
Monotonic rank association; robust to skew. Fare and SibSp: 0.45.

Zero is not independence

A perfect U-shape has r near 0.

A coefficient is one number describing a whole cloud: plot the cloud first

Categorical against numerical: compare distributions




```
(df.groupby('Survived')['Age']
   .agg(['count', 'mean', 'median', 'std']).round(2))
```

Output

Survived	count	mean	median	std
0	424	30.63	28.0	14.17
1	290	28.34	28.0	14.95

- Means differ by 2.3 years; medians are identical.
- Counts differ because 177 ages are missing.
- Age also varies by class and sex: confounding.

Grouped means alone hide overlap, skew and unequal spread

Categorical associations: effect size, not just p

```python
table = pd.crosstab(df['Pclass'], df['Survived'])
chi2, p, dof, _ = chi2_contingency(table)
n = table.to_numpy().sum()
v = np.sqrt(chi2 / (n * (min(table.shape) - 1)))
```

Output

```
Survived    0    1
Pclass
1          80  136
2          97   87
3         372  119

chi2 = 102.89, dof = 2
p = 4.55e-23, V = 0.340
```

102.89

chi-square, 2 d.o.f.

4.5e-23

p-value

0.340

Cramer's V

p answers

Is independence plausible?

V answers

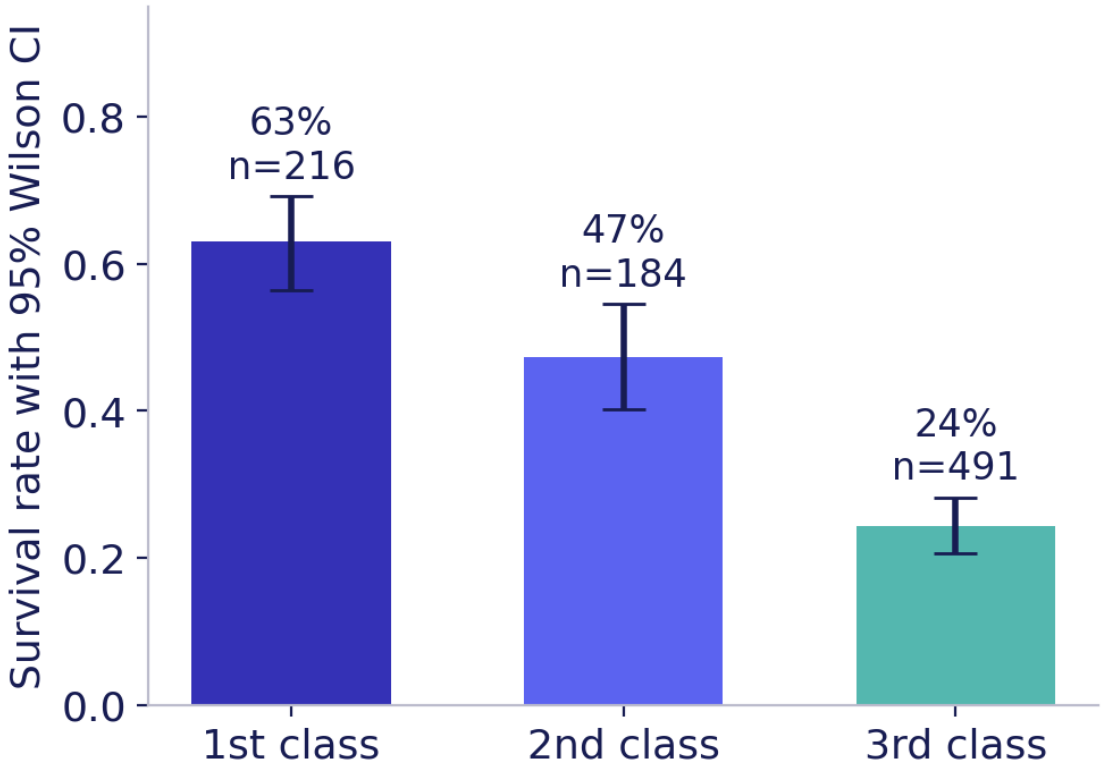
How strong is the association, from 0 to 1?

Neither answers

Why.

Report counts, conditional rates, p, effect size and the observational caveat together

Always attach uncertainty and group size



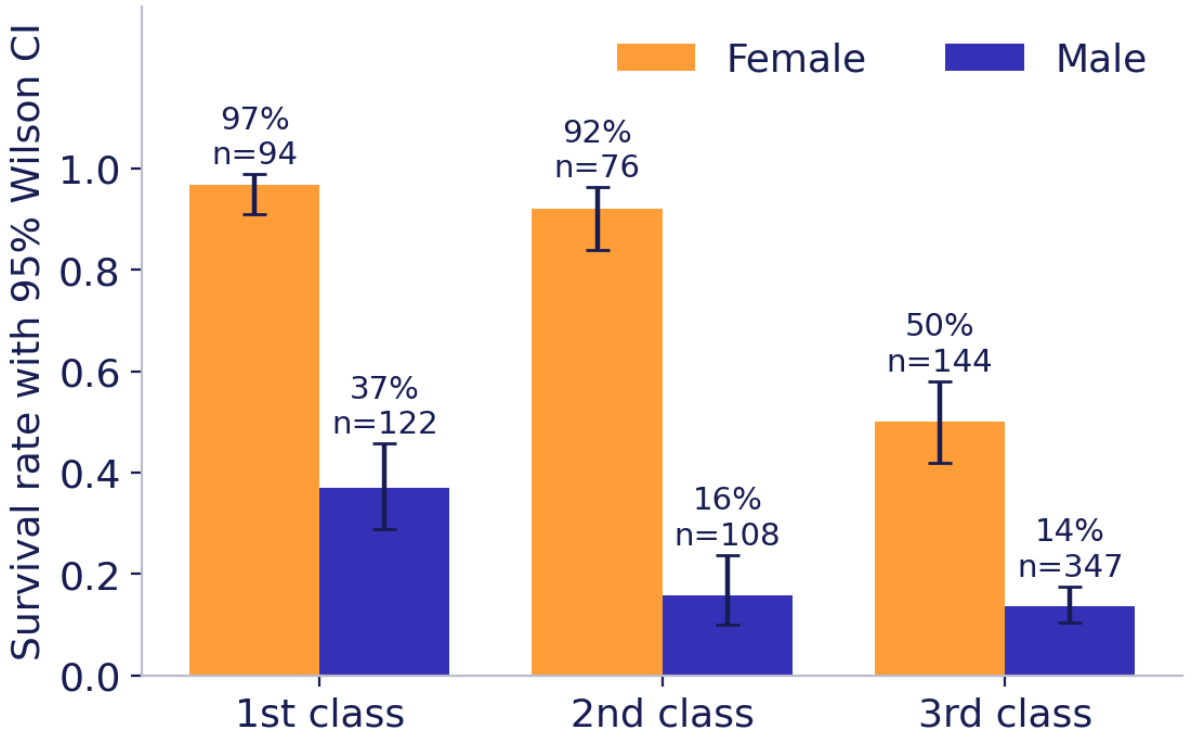

```python
from statsmodels.stats.proportion \
    import proportion_confint
lo, hi = proportion_confint(k, n, method='wilson')
```

- 3rd class: 24.2%, 95% CI 20.7% to 28.2% (n = 491).
- 2nd class: 47.3%, 95% CI 40.2% to 54.5% (n = 184).
- 1st class: 63.0%, 95% CI 56.3% to 69.1% (n = 216).
- Intervals do not overlap: the ordering is not sampling noise. Wilson intervals behave better than the textbook interval near 0 or 1.

A rate without a denominator and an interval is an anecdote

Multivariate EDA: does the pattern survive conditioning?

Persistence	Does it hold within groups?
Interaction	Does one feature change another's effect?
Composition	Do groups contain different populations?
Sparsity	Are intersections well supported?
Reversal	Can aggregation flip the direction?



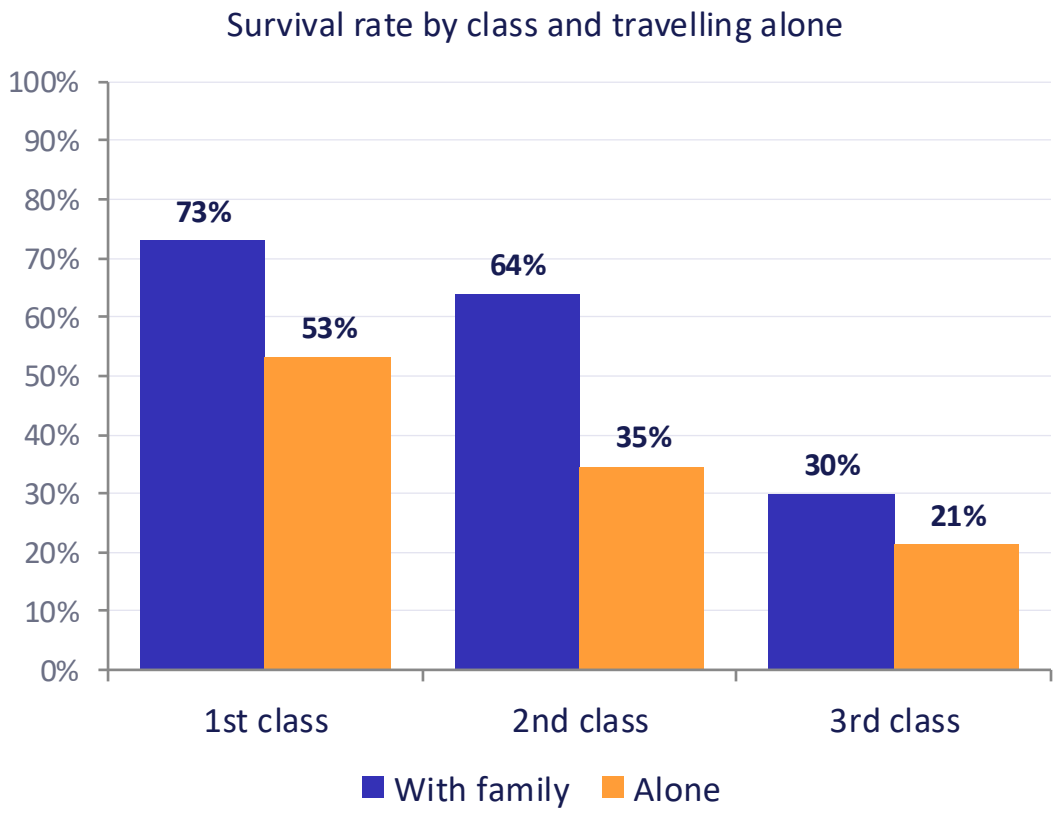
Test a few domain-motivated interactions, not every combination

Engineered features are hypotheses

```
df = df.assign(FamilySize=df.SibSp + df.Parch + 1)
df['IsAlone'] = (df.FamilySize == 1).astype(int)
(df.groupby(['Pclass', 'IsAlone']).Survived
   .agg(['count', 'mean']).round(3))
```

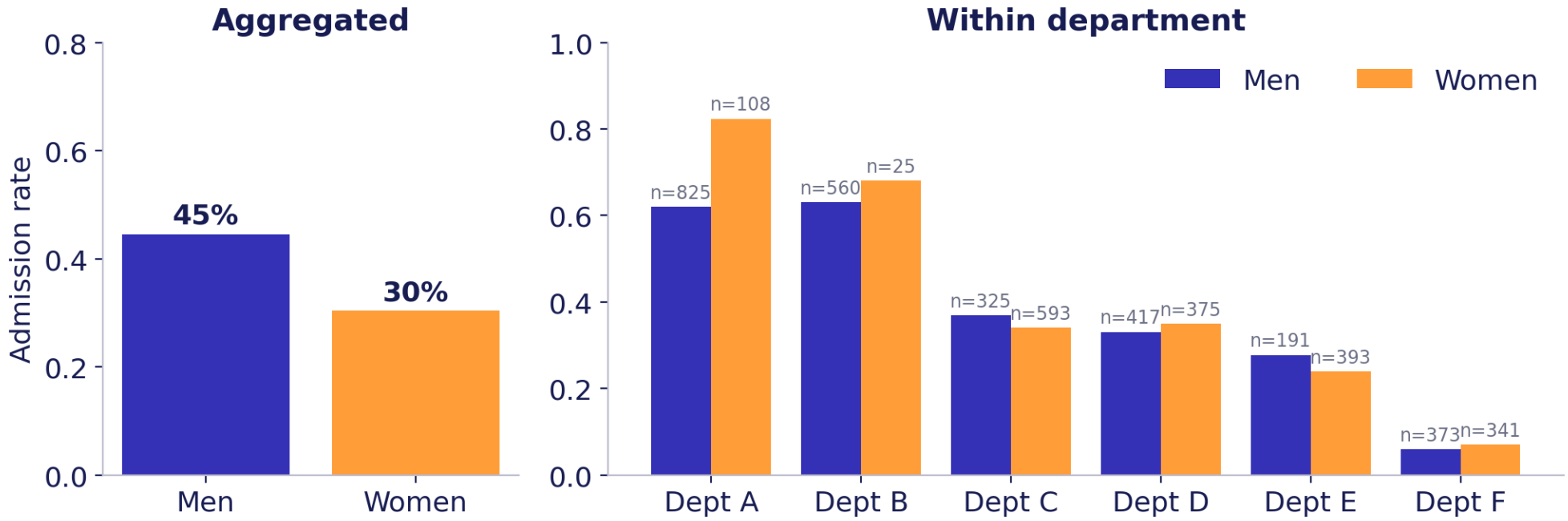
Output

Pclass	IsAlone	count	mean
1	0	107	0.729
	1	109	0.532
2	0	80	0.638
	1	104	0.346
3	0	167	0.299
	1	324	0.213



Check group counts before reading rates, and validate what the feature really measures

Simpson's paradox: the aggregate can reverse



Aggregated

Men 44.5% vs women 30.4% admitted (six largest departments).

Within department

Women's rate is higher in four of six. 93% of women applied to C to F, versus 49% of men.

The right aggregation depends on the causal question and the target population

Rows are not always independent

Repeated person or household
Same family, same latent traits

Time and seasonality
Neighbouring rows share structure

Location or organisation
Shared context and policies

Shared sensor or source
Shared measurement error

Uncertainty understated
Correlated rows carry less independent information.

Random splits leak
Related rows land in both train and test.

Poor generalisation
Performance may not carry to new groups or periods.

0.810 vs 0.800

Titanic random-forest CV accuracy: random folds vs folds grouped by ticket

Separate rows do not guarantee independent observations: name the grouping variable

4 Split, leak, preprocess

Learn parameters from training rows only, then package everything in a Pipeline.

Pause: find the leak in this workflow

```
df['Age'] = df['Age'].fillna(df['Age'].median())
X = StandardScaler().fit_transform(df[num])
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2)
model.fit(X_tr, y_tr)
print(model.score(X_te, y_te))
```

Think about: where is each learned quantity computed, and from which rows?

30 seconds

Which lines let information from the test rows influence training? Name every leak you can find.

Hint: the split must be the first data-dependent step, not the fourth

Split before learning from the data

1 Raw data › 2 Define groups or time › 3 Split train / test › 4 Fit on train only › 5 Apply to test unchanged

Safe to inspect before the split

Schema, dtypes and keys

File integrity and row counts

Field definitions and collection documentation

Rule violations (a count, not a fitted value)

Learn from training rows only

Imputation medians and modes

Scaling means and standard deviations

Category vocabularies

Feature selection and target encoding

The test set stands for future evidence: it never informs preprocessing decisions

Five ways evaluation becomes unrealistic

Target leakage	Field recorded after the outcome, e.g. a discharge code	› **Ask when each field becomes available**
Train-test contamination	Median or scaler fitted on all rows	› **Fit inside a Pipeline**
Temporal leakage	Random split of time-ordered data	› **Split by time**
Entity leakage	Same patient or ticket in train and test	› **Group-aware split**
Duplicate leakage	Identical or near-identical record on both sides	› **Compare keys and signatures**

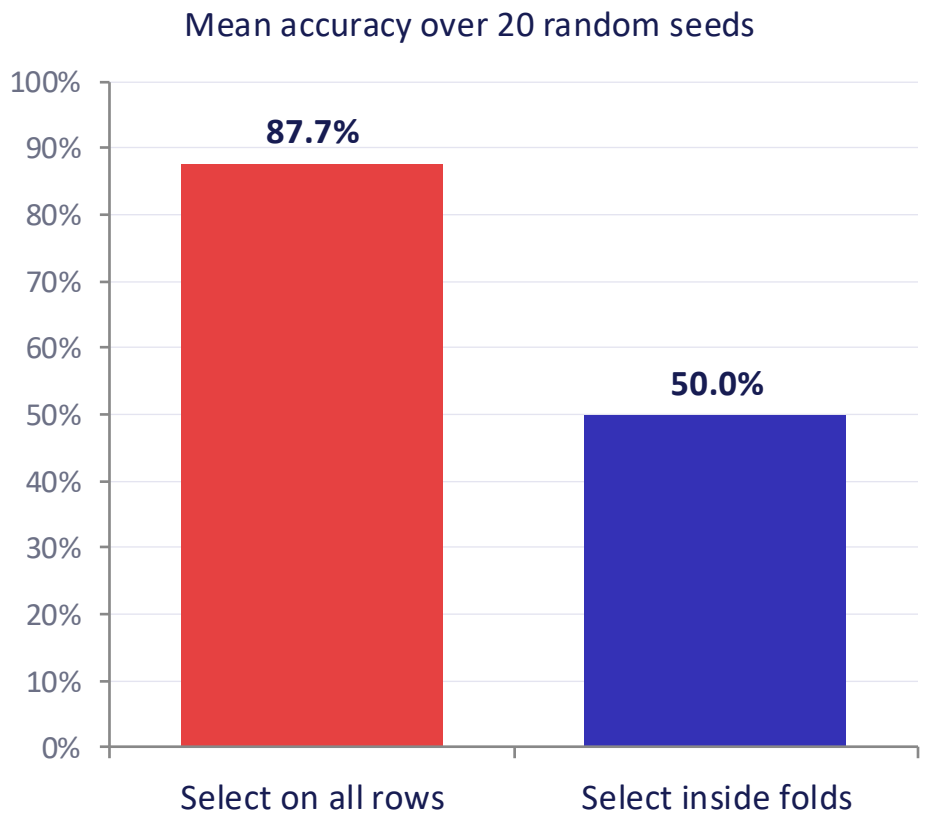
A high score is meaningless when the evaluation reproduces information that will not exist later

Leakage experiment: the labels are pure noise

```
X = rng.normal(size=(100, 5000))  # noise
y = rng.integers(0, 2, 100)       # random labels

# WRONG: select on all rows, then cross-validate
Xs = SelectKBest(f_classif, k=20).fit_transform(X, y)
cross_val_score(clf, Xs, y, cv=5)

# RIGHT: selection happens inside each training fold
pipe = Pipeline([('sel', SelectKBest(f_classif, k=20)),
                 ('clf', clf)])
cross_val_score(pipe, X, y, cv=5)
```



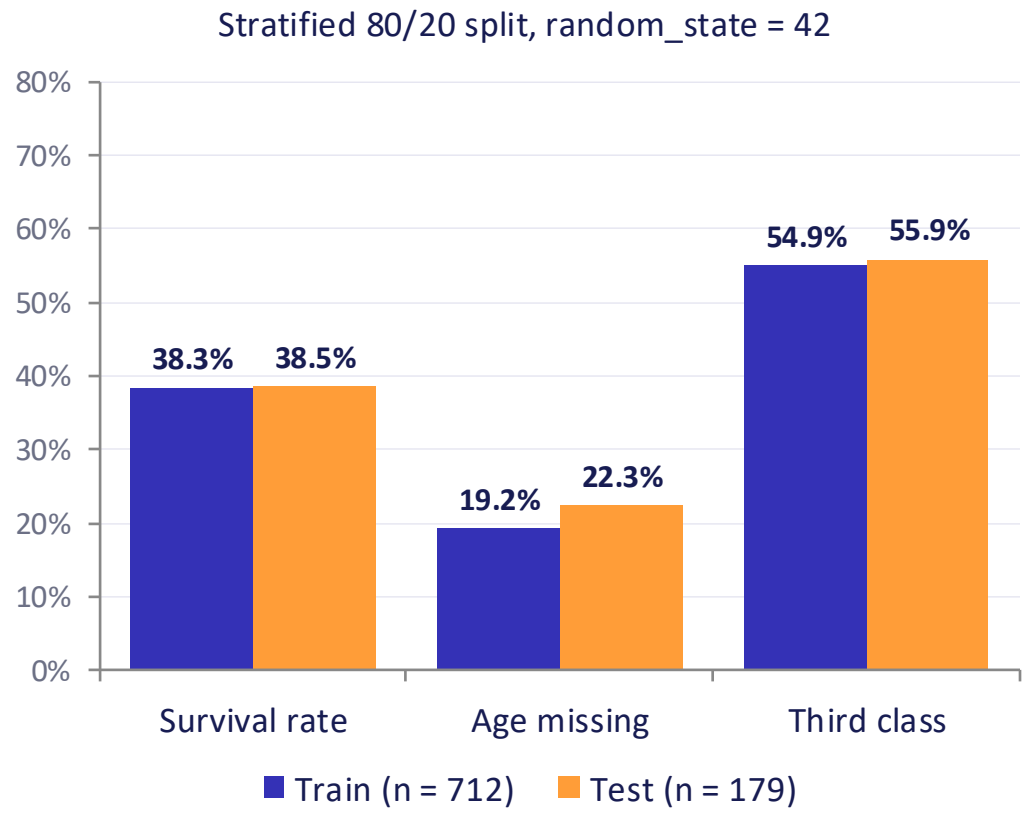
Random labels scored 88% under leakage; the honest pipeline reports chance, 50%

Compare training and test before trusting the score

```python
def profile(f):
    return pd.Series({
      'n': len(f),
      'survival': f['Survived'].mean(),
      'age_missing': f['Age'].isna().mean(),
      'median_fare': f['Fare'].median(),
      'third_class': f['Pclass'].eq(3).mean()})
pd.DataFrame({'train': profile(train),
              'test': profile(test)})
```

Sampling noise

With n = 179, one proportion has a standard error of about 3 to 4 points. A 3-point gap is not alarming.



A good split resembles deployment; matching distributions perfectly is not the goal

NumPy: vectorised, explicit, testable

```
fare = df['Fare'].to_numpy()
log_fare = np.log1p(fare)
q = np.quantile(fare, [0.25, 0.50, 0.75])
finite = np.isfinite(df['Age'].to_numpy())
print(q.round(2), finite.mean().round(3))
```

Output

```
[ 7.91 14.45 31.  ] 0.801

# 25% of fares are at most 7.91
# median 14.45; 80.1% of ages are finite
```

46x

faster than a Python loop for np.log1p on 1,000,000 values (0.245 s vs 0.005 s on one laptop)

- Shapes and dtypes are explicit.
- Boolean masks replace loops.
- Speed is not validity: justify why log1p suits Fare.

Vectorisation improves speed and makes transformations easier to inspect and test

Broadcasting: standardise a matrix in one line

X
shape (714, 2)

−

μ and σ
shape (2,) each

÷

Z
shape (714, 2)

The (2,) vectors are virtually repeated down all 714 rows: no loop, no copy.

```
X = df[['Age', 'Fare']].dropna().to_numpy()
mu = X.mean(axis=0)
sd = X.std(axis=0)
Z = (X - mu) / sd
print(X.shape, mu.shape)
print(Z.mean(axis=0).round(3))
print(Z.std(axis=0).round(3))
```

Output

```
(714, 2) (2,)
[ 0. -0.]
[1. 1.]

mu = [29.7  34.69]
sd = [14.52 52.88]

# -0. is floating-point noise
```

Fit μ and σ on training rows only; StandardScaler automates that pattern

Method chaining: one decision per line

```
analysis = (
  df.loc[:, ['Survived','Pclass','Sex','Age','Fare']]
    .assign(AgeMissing=lambda x: x['Age'].isna(),
            LogFare=lambda x: np.log1p(x['Fare']))
    .query('Fare >= 0')
    .groupby(['Pclass', 'Sex'], observed=True)
    .agg(passengers=('Survived', 'size'),
         survival_rate=('Survived', 'mean'),
         median_fare=('Fare', 'median'))
    .reset_index()
)
```

Reading the chain

select, derive, filter, group, aggregate, reshape.

Each step is one visible analytical decision that a reviewer can question.

Output

	Pclass	Sex	passengers	survival_rate	median_fare
0	1	female	94	0.968	82.66
1	1	male	122	0.369	41.26
2	2	female	76	0.921	22.00
3	2	male	108	0.157	13.00
4	3	female	144	0.500	12.48
5	3	male	347	0.135	7.92

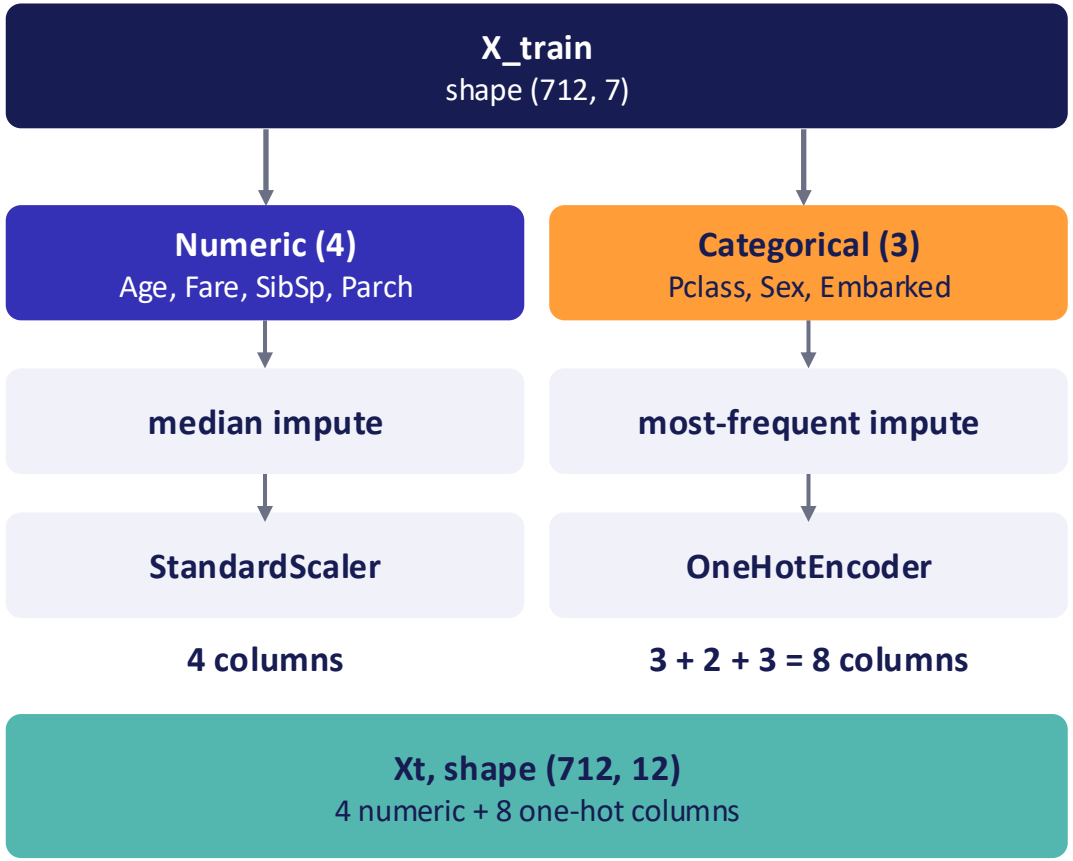
A readable chain documents selection, transformation, filtering and aggregation

ColumnTransformer: role-specific preprocessing

```
numeric = ['Age', 'Fare', 'SibSp', 'Parch']
categorical = ['Pclass', 'Sex', 'Embarked']

preprocess = ColumnTransformer([
  ('num', Pipeline([
    ('impute', SimpleImputer(strategy='median')),
    ('scale', StandardScaler())]), numeric),
  ('cat', Pipeline([
    ('impute', SimpleImputer(
        strategy='most_frequent')),
    ('encode', OneHotEncoder(
        handle_unknown='ignore'))]), categorical)])

Xt = preprocess.fit_transform(X_train)
print(X_train.shape, Xt.shape)  # (712,7) (712,12)
```

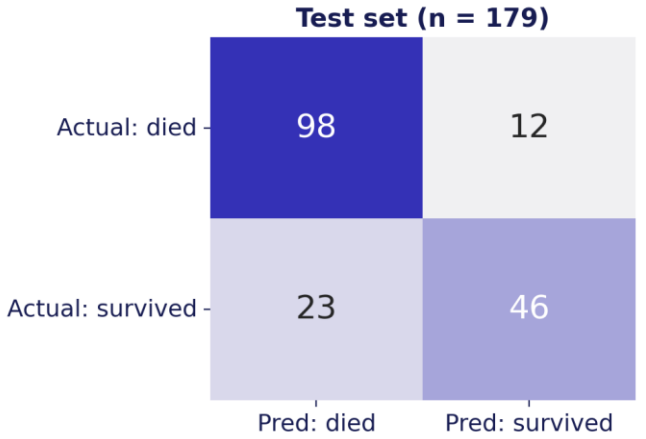


One fitted object learns medians, scales and category vocabularies from X_train only

A Pipeline makes leakage hard, then evaluate honestly

Fit preprocessing › **Transform train** › **Fit classifier** › **Score untouched test**

```
model = Pipeline([
  ('prep', preprocess),
  ('clf', LogisticRegression(max_iter=1000))])
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
# 0.804
```



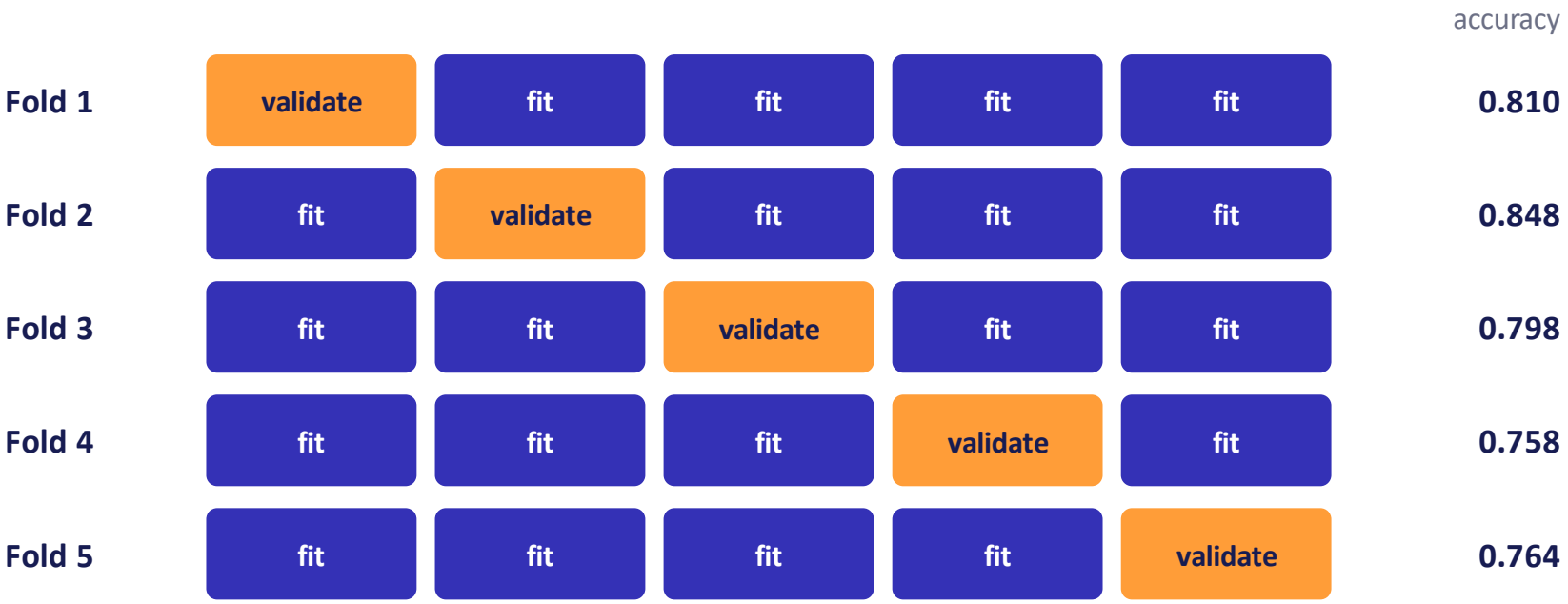
0.804
test accuracy

0.615
majority baseline

0.667
recall on survivors

Accuracy beats the baseline, yet 23 of 69 survivors were missed: read the confusion matrix

Cross-validation refits everything in every fold



0.796

mean accuracy, SD 0.033 across the five folds

```
cv = StratifiedKFold(5, shuffle=True, random_state=0)
scores = cross_val_score(model, X, y, cv=cv)
```

Why it matters

Each fold fits imputer, scaler, encoder and classifier on its own training part. Use GroupKFold when rows share an entity.

Report the spread across folds, not only the mean: 0.76 to 0.85 on the same data

Automated profiling is a starting point

Useful for

A fast inventory of schema and summaries

Missingness and frequency tables

Candidate questions to investigate

Repeatable first-pass checks

Cannot decide

Semantic correctness of a field

Causal interpretation

Ethical relevance of a feature

When a feature becomes available

Whether deployment matches the sample

Example: a profiler reports PassengerId as unique with high cardinality. It cannot tell you whether that is an identifier, a sequence proxy, or a valid predictor. Tools: df.describe(include='all'), ydata-profiling, Sweetviz.

Automated reports find patterns; analysts decide which ones matter and why

A reproducible EDA report traces actions to evidence

1
Context
Task, population, unit, source

2
Structure
Schema, roles, keys, dimensions

3
Quality
Missing, invalid, duplicate, extreme

4
Patterns
Groups, interactions, split checks

5
Actions
Pipeline rules, monitoring, risks

Traceability in practice

Finding
Age missing: 19.9% overall, 27.7% in 3rd class

›

Decision
Median imputation inside the pipeline, plus a missing-indicator

›

Limitation
MNAR cannot be excluded; imputed SD is understated

Keep code, dataset version, environment and random seeds with the report.

A notebook that runs only in the author's current session is not reproducible

Case study: from question to decision

Question	Measure	Evidence	Limits	Decision
Does class relate to survival once sex is considered?	Survival rate by class and sex, with n and Wilson CI	Women 97%, 92%, 50%; men 37%, 16%, 13.5% (1st, 2nd, 3rd)	Observational. Class also tracks age, deck and rescue procedure	Keep Pclass and Sex categorical; evaluate by subgroup; monitor class mix

A useful EDA result ends in an action or an explicit unresolved risk

Common EDA mistakes: all of them run without errors

Plots with no question, denominator or interpretation	Reasoning
Treating storage dtypes as variable types	Reasoning
Dropping rows or outliers before measuring them	Process
Reporting only the favourable relationships	Process
Fitting imputation, scaling or selection on all data	Design
Reading association or importance as causation	Reasoning
Ignoring row dependence, time order or entity overlap	Design

Before accepting a result ask: which question, which denominator, which assumption, what alternative, what decision?

Lab 5: ten short tasks, all done in class

What you will do

1. Tasks 1 to 3: data contract, missing values, extreme values
2. Tasks 4 and 5: ECDF, baseline, effect size and intervals
3. Task 6: condition on a second variable; Simpson's paradox
4. Tasks 7 and 8: split, leak-free scaling, the noise experiment
5. Tasks 9 and 10: ColumnTransformer, Pipeline, cross-validation

How we will work

- ✓ Most code is written: fill in a few blanks
- ✓ A check cell prints PASS or CHECK AGAIN
- ✓ Stuck? Read the getting-started guide first
- ✓ Short written answers: one or two sentences

Summary: six ideas to keep

1

Context first

Unit of analysis and semantic roles define what the data means.

2

Measure quality

Completeness, validity, uniqueness, consistency and timeliness.

3

Match method to question

Variable types and questions select the display and statistic.

4

Condition and quantify

Counts, intervals and subgroups; beware Simpson's paradox.

5

Split first

Leakage, dependence and shift can invalidate correct code.

6

Findings become rules

Each finding is a pipeline rule, a monitor or a documented limitation.

Next week: turn the EDA decisions into a reproducible ETL or ELT pipeline

Reading list

Tukey (1977)	Exploratory Data Analysis. Addison-Wesley.
Anscombe (1973)	Graphs in statistical analysis. The American Statistician, 27(1).
Little and Rubin (2019)	Statistical Analysis with Missing Data, 3rd ed. Wiley.
Kapoor and Narayanan (2023)	Leakage and the reproducibility crisis in ML-based science. Patterns, 4(9).
Gebru et al. (2021)	Datasheets for datasets. Communications of the ACM, 64(12).
scikit-learn user guide	Common pitfalls and recommended practices.

Start with Anscombe (2 pages), the scikit-learn pitfalls page and section 7.10.2 of ESL